

Graph Algorithms

Sutanu Gayen

August 2024

1 Depth First Search

1.1 Pseudocode

Algorithm 1: DFS($G, s, start, finish, \pi, color, clock$)

Input: The adjacency list of a (Directed) graph $G = (V, E)$, A source vertex s , Four arrays $start$, $finish$, π , $color$ each indexed by $v \in V$, An integer $clock$

Output: None. The arrays and clock will be updated.

```
1  $S \leftarrow$  an empty stack of MAX value  $2(|V| + |E|)$ 
2  $S.Push(s)$ 
3 while  $S$  is not empty do
4    $u \leftarrow S.Peek()$ 
5   if  $color[u] = white$  then
6      $start[u] \leftarrow clock$ 
7      $clock++$ 
8      $color[u] \leftarrow gray$ 
9     for each  $(u, w) \in E$  do
10      if  $color[w] = white$  then
11         $S.Push(w)$ 
12         $\pi[w] \leftarrow u$ 
13         $// (u, w)$  is a tree/forward edge
14      else if  $color[w] = gray$  then
15         $// (u, w)$  is a backward edge
16      else if  $color[w] = black$  then
17         $// (u, w)$  is a cross/forward edge
18    else if  $color[u] = gray$  then
19       $color[u] \leftarrow black$ 
20       $finish[u] \leftarrow clock$ 
21       $clock++$ 
22       $S.pop()$ 
23    else if  $color[u] = black^a$  then
24       $S.pop()$ 
```

^aI missed this check in class. You must do this to take care of the case when a vertex appears more than once in the stack.

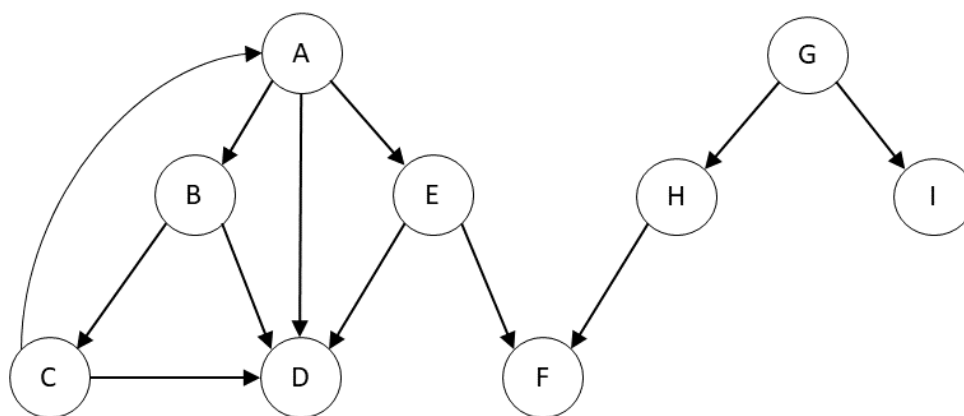


Figure 1: Input Graph

Algorithm 2: DFS-EXPLORE(G)

Input: The adjacency list of a (Directed) graph $G = (V, E)$

Output: Three arrays $start, finish, \pi$ each of size $|V|$

```

1  $clock \leftarrow 0$ 
2  $start \leftarrow$  all  $-1$  array of size  $|V|$ 
3  $finish \leftarrow$  all  $-1$  array of size  $|V|$ 
4  $\pi \leftarrow$  all  $NULL$  array of size  $|V|$ 
5  $color \leftarrow$  all  $white$  array of size  $|V|$ 
6 for each  $v \in V$  do
7   if  $start[v] = -1$  then
8      $DFS(G, v, start, finish, \pi, color, clock)$ 
9 return ( $start, finish, \pi$ )

```

Notes.

1. This is a modification of the basic DFS with two important differences:

- (a) children are popped before the parent is popped
- (b) first and last time seeing are timestamped

Once you keep these two facts in mind, you can develop the pseudocode with a little creativity.

2. It's possible that a vertex is pushed more than once into the stack. Only the topmost pushing will go through line numbers 5-14. This is also according to the DFS logic that we go deeper and deeper and then backtrack. For any subsequent presence in the stack, line numbers 20-21 will simply throw out the vertex without doing anything.

Nevertheless, the combination of while and for loop (Line numbers 10-14) at most run once for each edge. Thus the final running time is still $O(n + m)$.

3. π value at line number 12 may keep on updating. Thus tree edges can only be identified after DFS completes. Final value of π will be set via the deepest path which is according to the DFS logic.

1.2 Exercise

Question: Consider the graph in Figure 1.

1. Write the adjacency list of this graph. A alphabetically higher ordered vertex needs to be written later in the list.
2. Suppose you are given as input the adjacency list from previous question. You run DFS-EXPLORE on this graph. Write down the start, finish, and π array that you are going to get as a result. You must push a alphabetically lower ordered child earlier.
3. Using answer to previous question, draw the dfs tree, the dfs timeline, and perform edge classification

Answer:

1.

```

A  B  D  E
B  C  D
C  A  D
D
E  D  F
F
G  H  I
H  F

```

2.

vertex	start	finish	π
A	0	11	NULL
B	7	10	A
C	8	9	B
D	4	5	E
E	1	6	A
F	2	3	E
G	12	17	NULL
H	15	16	G
I	13	14	G

Table 1: Returned Values from Explore

3.

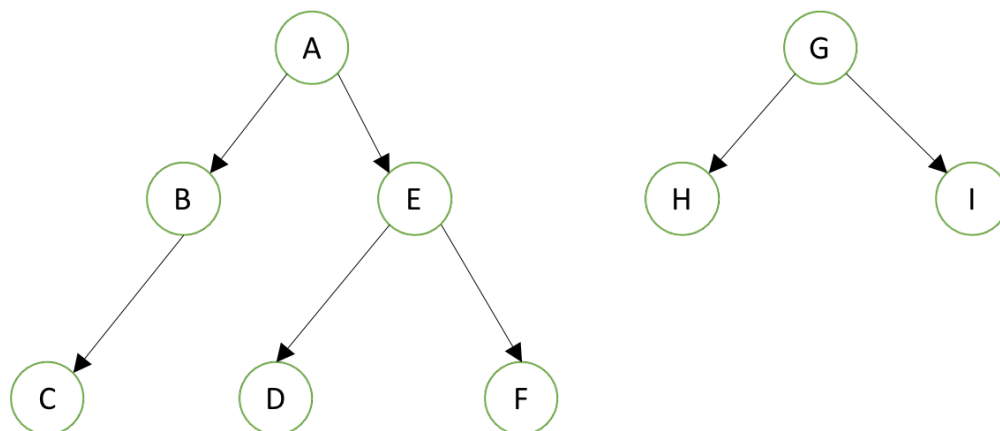


Figure 2: DFS tree

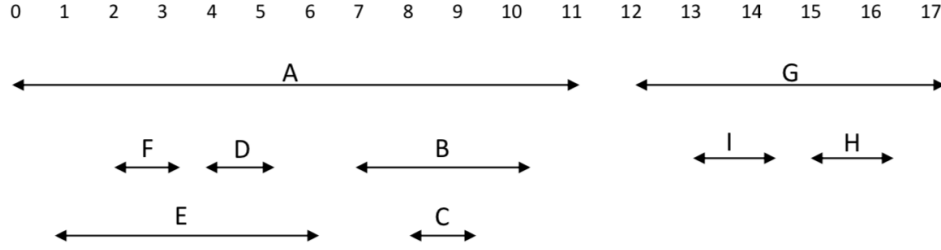


Figure 3: DFS timeline

Edge classification:

- Tree edges: given above
- Forward edges: (A, D)
- Backward edges: (C, A)
- Cross edges: $(C, D), (H, F), (B, D)$

2 Dijkstra's Algorithm

2.1 Pseudocode

Algorithm 3: SSSP(G, s)

Input: The weighted adjacency list of a (Directed) graph $G = (V, E)$, A source vertex s . Weight of an edge (u, v) is given by $wt(u, v)$.

Output: Shortest Path tree π

```

1  $\pi \leftarrow$  an all NULL array of size  $|V|$ 
2  $PQ \leftarrow$  A priority queue having an item for each  $v \in V$  with  $PQ.key(v) = \infty$ 
3  $DecreaseKey(PQ, s, 0)$ 
4 for  $i = 1$  to  $n$  do
5    $u \leftarrow ExtractMin(PQ)$ 
6   for each  $(u, w) \in E$  do
7     if  $PQ.key(u) + wt(u, w) < PQ.key(w)$  then
8        $DecreaseKey(PQ, w, PQ.key(u) + wt(u, w))$ 
9        $\pi(w) \leftarrow u$ 

```

2.2 Exercise

Question Consider the graph in Figure 4.

1. Write down the weighted adjacency list of this graph. You must write an alphabetically lower vertex first, both in the rows and columns. After writing each child, write the weight of the corresponding edge.
2. Write down the states of the priority queue (mapping, heap, and key values) and π array before the first iteration and also after each iteration.

Each item in the heap will contain a (vertex, key) tuple. Vertex will be an index of the mapping array. The heap operations will be carried out w.r.t. the key. Carefully follow the pseudocodes for extract-min and decrease-key. In case of a tie between two keys, put the alphabetically smaller vertex first in the heap array.

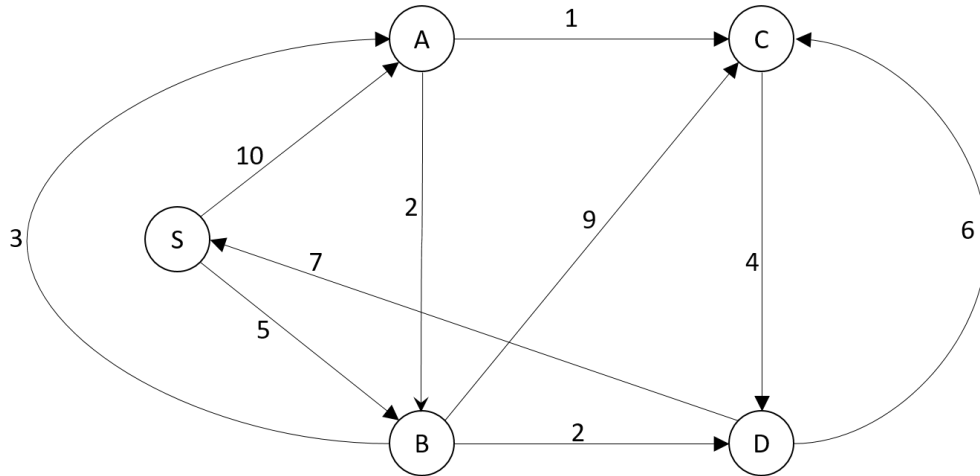


Figure 4: Input Graph for Single Source Shortest Paths

The mapping array will be indexed by the vertices. Each entry of this array will be the location of the vertex within the heap. Vertices that have been extracted from the heap will have empty entries in the mapping array.

The key array will contain the current key value of each vertex. Keys for active vertices in the heap can be found by suitably combining information from the heap array and the mapping array. Note that the key of a vertex will remain frozen once it is extracted out of the heap.

3. Based on the answer to the previous question, draw the shortest path tree labeled with edge weights.

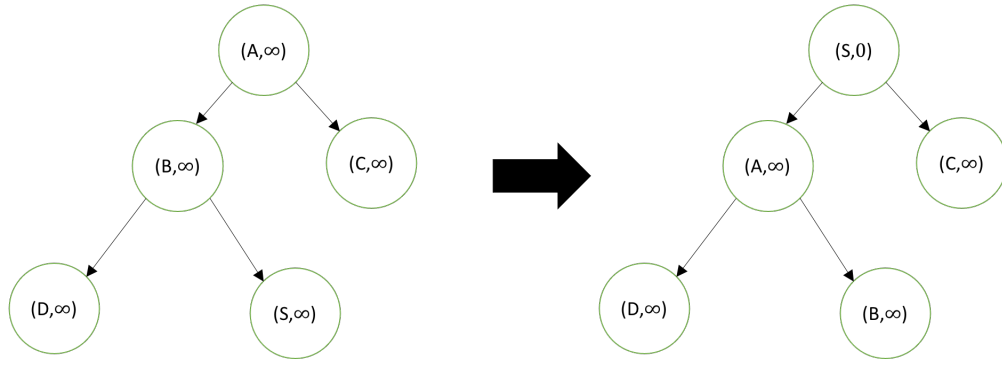
Answer

- 1.

A	B	2	C	1		
B	A	3	C	9	D	2
C	D	4				
D	C	6	S	7		
S	A	10	B	5		

2. Before first iteration

- We first make all keys to be ∞ . Note we are writing an alphabetically earlier vertex first in the heap in case of a tie.
- Then S's key will be decreased to 0 at Line # 3.



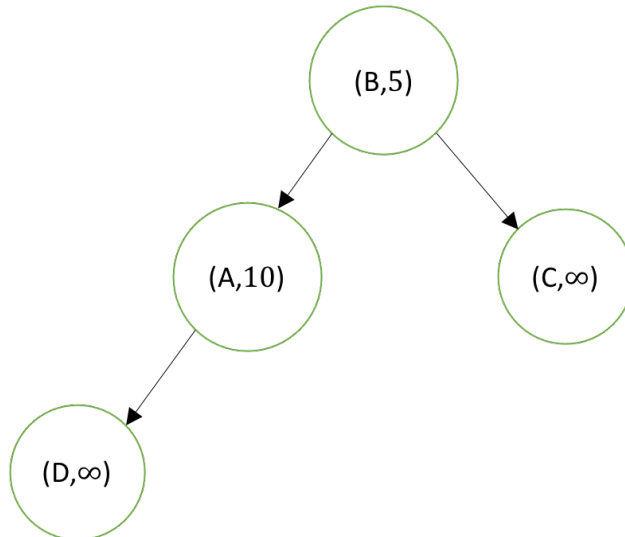
	1	2	3	4	5
Heap	(S, 0)	(A, ∞)	(C, ∞)	(D, ∞)	(B, ∞)

	A	B	C	D	S
mapping	2	5	3	4	1
key	∞	∞	∞	∞	0

	A	B	C	D	S
π	NULL	NULL	NULL	NULL	NULL

After first iteration

- S will be extracted
- First A's key will be decreased to 10 (note that in S's adjacency list, A comes first)
- Then B's key will be decreased to 5
- S will be the parent of A and B



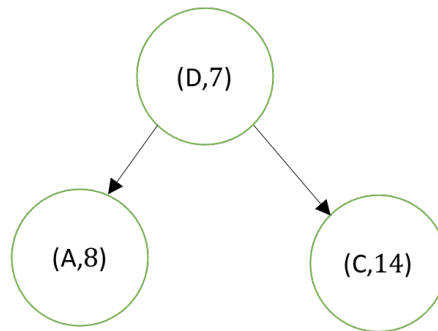
	1	2	3	4
Heap	(B, 5)	(A, 10)	(C, ∞)	(D, ∞)

	A	B	C	D	S
mapping	2	1	3	4	–
key	10	5	∞	∞	0

	A	B	C	D	S
π	S	S	NULL	NULL	NULL

After second iteration

- B will be extracted
- A's key will be decreased to 8
- C's key will be decreased to 14
- D's key will be decreased to 7
- B will be the parent of A, C, D



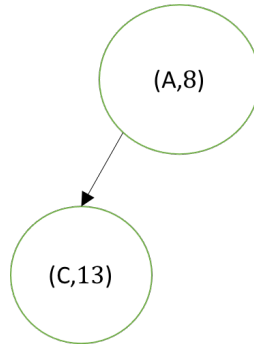
	1	2	3
Heap	(D, 7)	(A, 8)	(C, 14)

	A	B	C	D	S
mapping	2	–	3	1	–
key	8	5	14	7	0

	A	B	C	D	S
π	B	S	B	B	NULL

After third iteration

- D will be extracted
- C's key will be decreased to 13
- D will be C's parent



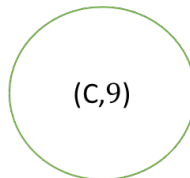
	1	2
Heap	(A, 8)	(C, 13)

	A	B	C	D	S
mapping	1	–	2	–	–
key	8	5	13	7	0

	A	B	C	D	S
π	B	S	D	B	NULL

After fourth iteration

- A will be extracted
- C's key will be decreased to 9
- C's parent will be A



	1
Heap	(C, 9)

	A	B	C	D	S
mapping	–	–	1	–	–
key s	8	5	9	7	0

	A	B	C	D	S
π	B	S	A	B	NULL

After fifth iteration

- C will be extracted. Thus, the heap will be empty.
- This is the last iteration. No more decrease is possible.

	A	B	C	D	S
mapping	–	–	–	–	–
key	8	5	9	7	0

	A	B	C	D	S
π	B	S	A	B	NULL

3.

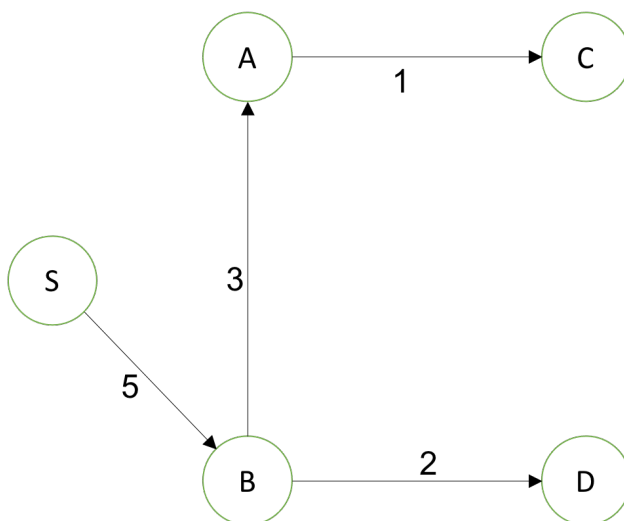


Figure 5: Single Source Shortest Path Tree. Note the agreement with final “key” array.

3 Further Practice Problems

- Try out the previous exercises with some other graphs that you’ll choose.
- Conduct a similar exercise with the BFS pseudocode.
- You may also try out relevant chapter exercises from Dasgupta and Cormen.